INF560 / Unidad 3 / **GL11**

GUÍA DE LABORATORIO 11

Carga de imágenes en los eventos

File Storage de Laravel: subir, validar, mostrar, reemplazar y eliminar archivos

Desarrollo Web Backend · 02/2026 · eventos-app · Laravel 13 · Tailwind v4

Docente: M. Sc. Huáscar Fedor Gonzales Guzmán · Carrera de Ingeniería Informática · UATF

MÉTODOS HTTP QUE INTERVIENEN

GET ver imagen POST subir PUT reemplazar DELETE limpiar

01 Datos de la guía

Campo	Detalle
Asignatura	INF560 — Desarrollo Web Backend (5.º semestre)
Unidad	Unidad 3 — Frameworks backend con Laravel
Proyecto	eventos-app (acumulativo) — repositorio `INF560-2026-02-Apellido`
Prerrequisito	GL10 — Autorización (políticas `EventoPolicy`, rol `admin`) funcionando
Duración estimada	6 horas académicas (una sesión de laboratorio + trabajo independiente)
Entregable	Repositorio actualizado con un commit por ejercicio + trabajo independiente

Objetivo general

Incorporar a **eventos-app** la carga de una imagen de portada por evento, usando el sistema de archivos (File Storage) de Laravel, de modo que el archivo se suba de forma segura, se almacene en el disco `public`, se muestre en la cartelera y en el detalle, y se elimine cuando ya no se necesita.

Objetivos específicos

- Explicar el recorrido de un archivo: formulario `multipart/form-data` → `UploadedFile` → disco → ruta en la base de datos → URL pública.
- Configurar el disco `public` y el enlace simbólico `public/storage` con `php artisan storage:link`.
- Validar archivos con las reglas `image`, `mimes` y `max`, entendiendo los límites que impone PHP.
- Guardar únicamente la **ruta relativa** del archivo en la columna `imagen` y construir la URL con un accessor de Eloquent.
- Reemplazar, quitar y eliminar imágenes sin dejar archivos huérfanos en el servidor.
- Comprobar el comportamiento con pruebas automatizadas usando `Storage::fake()` y `UploadedFile::fake()`.

02 Punto de partida

Esta guía continúa exactamente desde el último commit del proyecto de clase (`Agrega autenticacion y autorizacion`). Antes de empezar, compruebe que su proyecto tiene lo siguiente; buena parte del trabajo ya está preparado y solo falta conectarlo:

Elemento	Estado actual en eventos-app	¿Qué haremos?
Migración `create_eventos_table`	Ya existe la columna `imagen` (`string`, `nullable`)	Nada: la reutilizamos
Modelo `Evento`	`imagen` ya está en `$fillable`	Agregar el accessor `imagen_url`
Formulario `partials/evento-form`	No tiene campo de archivo	Agregar campo, vista previa y opción «Quitar»
`create` / `edit`	Formularios sin `enctype`	Agregar `enctype="multipart/form-data"`

Elemento	Estado actual en eventos-app	¿Qué haremos?
`EventoController`	No procesa archivos	Validar y guardar en `store`/`update`; limpiar en `destroy`
`evento-card` y `show`	Ya muestran `<img src="{{ $imagen }}">` con respaldo SVG	Pasarles la URL pública en lugar de la ruta
`EventoPolicy`	Solo el admin crea; solo el dueño edita/elimina	Nada: protege también la carga

terminal Copiar

```
# Sincronice su repositorio y levante el proyecto
$ git pull
$ composer install
$ php artisan migrate
$ composer run dev
```

03 Fundamentos breves

¿Cómo viaja un archivo?

Un formulario HTML normal envía sus datos como texto (`application/x-www-form-urlencoded`). Para enviar archivos binarios el formulario debe declararse como `multipart/form-data`; el navegador divide la petición en «partes» y una de ellas contiene los bytes de la imagen. Laravel recibe esa parte como un objeto `Illuminate\Http\UploadedFile`, que vive en una carpeta temporal hasta que lo movemos a un **disco**.

Paso	Dónde ocurre	Resultado
1. Selección	Navegador — `<input type="file">`	El usuario elige `afiche.jpg`
2. Envío	`POST /eventos (multipart/form-data)`	Los bytes viajan junto al resto de campos
3. Validación	`$request->validate()`	Se comprueba que sea imagen y su tamaño
4. Almacenamiento	`->store('eventos', 'public')`	Archivo en `storage/app/public/eventos/Xa9…jpg`
5. Persistencia	Columna `eventos.imagen`	Se guarda el texto `eventos/Xa9…jpg`
6. Visualización	`GET /storage/eventos/Xa9…jpg`	El servidor web entrega el archivo estático

DEFINICIÓN

Disco (disk): una ubicación de almacenamiento configurada en `config/filesystems.php`. Laravel ofrece la misma API (`Storage::disk(...)`) para un disco local, uno público o un bucket S3, de modo que cambiar de almacenamiento no obliga a reescribir el código.

Disco	Carpeta real	¿Accesible desde el navegador?
`local`	`storage/app/private`	No. Para archivos privados (facturas, respaldos).

Disco	Carpeta real	¿Accesible desde el navegador?
`public`	`storage/app/public`	Sí, a través del enlace `public/storage`.
`s3`	Bucket en la nube	Según la configuración del bucket (Unidad 5).

> **NOTA**
> En la base de datos **nunca** se guarda la imagen; se guarda su **ruta relativa** al disco (`eventos/archivo.jpg`). La URL completa se calcula al mostrarla. Así, si mañana el proyecto pasa a S3 o cambia de dominio, los registros no se tocan.

Reglas de validación para archivos

Regla	Qué comprueba	Ejemplo
`image`	Que sea jpg, jpeg, png, bmp, gif o webp (desde Laravel 12 excluye SVG)	`'image'`
`mimes:…`	Restringe las extensiones según el contenido real del archivo	`'mimes:jpg,jpeg,png,webp'`
`max:…`	Tamaño máximo en **kilobytes**	`'max:2048'` (2 MB)
`dimensions:…`	Ancho/alto mínimos o máximos, proporción	`'dimensions:min_width=600'`

04 Ejercicio 1 — Preparar el disco público

El disco `public` guarda los archivos en `storage/app/public`, pero el servidor web solo expone la carpeta `public/`. El comando `storage:link` crea un enlace simbólico `public/storage → storage/app/public` para «abrir una ventana» a esa carpeta.

1. Cree el enlace simbólico desde la raíz del proyecto:

```
terminal                                                    Copiar
$ php artisan storage:link

   INFO  The [public/storage] link has been connected to [storage/app/public].
```

2. Verifique que el enlace existe. En Linux/macOS use `ls -l public`; en Windows, el Explorador mostrará `public/storage` con ícono de acceso directo.
3. Abra `.gitignore` y confirme que contiene la línea `/public/storage`. El enlace **no se sube** al repositorio: cada persona que clone el proyecto debe ejecutar `storage:link` una vez.
4. Revise en su `.env` que `APP_URL` coincida exactamente con la dirección que usa en el navegador (por ejemplo `http://localhost:8000`). Laravel usa ese valor para construir las URLs de las imágenes.

VERIFICACIÓN

Cree un archivo de prueba y ábralo en el navegador:

`echo hola > storage/app/public/prueba.txt` → visite `http://localhost:8000/storage/prueba.txt`. Si ve «hola», el disco público funciona. Después elimine `prueba.txt`.

NOTA

Agregue al `README.md`, en la sección **Instalación**, el paso `php artisan storage:link` para que sus compañeros no se encuentren con imágenes rotas al clonar el proyecto.

commit Copiar

```
$ git add README.md
$ git commit -m "Agrega storage:link a las instrucciones de instalacion"
```

05 Ejercicio 2 — Campo de archivo en el formulario

2.1 Habilitar el envío de archivos

Abra `resources/views/eventos/create.blade.php` y agregue el atributo `enctype` a la etiqueta `<form>`:

resources/views/eventos/create.blade.php Copiar

```
<form action="{{ route('eventos.store') }}" method="post" enctype="multipart/form-data"
    class="overflow-hidden rounded-2xl border border-slate-200 bg-white shadow-sm">
    @csrf
```

Haga lo mismo en `resources/views/eventos/edit.blade.php`:

resources/views/eventos/edit.blade.php Copiar

```
<form action="{{ route('eventos.update', $evento) }}" method="post" enctype="multipart/form-data"
    class="overflow-hidden rounded-2xl border border-slate-200 bg-white shadow-sm">
    @csrf
    @method('PUT')
```

ADVERTENCIA

Si olvida `enctype="multipart/form-data"`, el formulario se envía sin error, pero el archivo **nunca llega**: `$request->hasFile('imagen')` devuelve `false` y el evento se guarda sin imagen. Es el error más frecuente de esta guía.

NOTA

PHP solo interpreta cuerpos `multipart` en peticiones **POST**. Por eso el formulario de edición usa `method="post"` y la directiva `@method('PUT')`: Laravel recibe un POST (con el archivo) y lo trata como PUT gracias al campo oculto `_method`.

2.2 Agregar el campo en el formulario parcial

En `resources/views/partials/evento-form.blade.php`, dentro de la sección **Datos del evento**, inserte el siguiente bloque inmediatamente **después** del campo `descripcion` (antes de la grilla de fecha y lugar):

resources/views/partials/evento-form.blade.php Copiar

```
{{-- Imagen de portada --}}
<div>
    <label for="imagen" class="block text-sm font-medium text-slate-700">Imagen de portada</label>

    {{-- Solo en edición: muestra la imagen actual y permite quitarla --}}
    @if ($evento?->imagen_url)
        <div class="mt-2 flex items-center gap-4 rounded-lg border border-slate-200 bg-slate-50 p-
3">
            <img src="{{ $evento->imagen_url }}" alt="Imagen actual"
                class="h-20 w-32 rounded-md object-cover">
            <label class="flex cursor-pointer items-center gap-2 text-sm text-slate-600">
                <input type="checkbox" name="quitar_imagen" value="1"
                    class="h-4 w-4 accent-rose-600" @checked(old('quitar_imagen'))>
                Quitar la imagen actual
            </label>
        </div>
    @endif

    <input type="file" id="imagen" name="imagen" accept="image/jpeg,image/png,image/webp"
        class="mt-2 block w-full cursor-pointer text-sm text-slate-600
               file:mr-4 file:rounded-lg file:border-0 file:bg-blue-50 file:px-4 file:py-2
               file:text-sm file:font-medium file:text-blue-700 hover:file:bg-blue-100">
    <p class="mt-1.5 text-xs text-slate-400">JPG, PNG o WEBP. Máximo 2 MB.</p>

    {{-- Vista previa de la imagen recién elegida --}}
    <img id="vista-previa" src="" alt="Vista previa"
        class="mt-3 hidden h-48 w-full rounded-lg object-cover ring-1 ring-slate-200">

    @error('imagen')
        <p class="mt-1.5 text-sm text-rose-600">{{ $message }}</p>
    @enderror
</div>
```

La variante `file:` de Tailwind v4 estiliza el botón nativo «Elegir archivo», y el atributo `accept` hace que el explorador del sistema operativo muestre solo imágenes. Ojo: `accept` es una ayuda visual, **no** una medida de seguridad; la validación real ocurre en el servidor.

2.3 Vista previa con JavaScript

Al final del mismo archivo parcial, agregue un pequeño script que muestre la imagen elegida antes de enviar el formulario. `URL.createObjectURL()` crea una URL temporal en memoria; no sube nada al servidor.

resources/views/partials/evento-form.blade.php (al final) Copiar

```
<script>
    document.getElementById('imagen').addEventListener('change', (e) => {
        const archivo = e.target.files[0];
        const vista = document.getElementById('vista-previa');

        if (!archivo) {
            vista.classList.add('hidden');
            return;
        }

        vista.src = URL.createObjectURL(archivo);
        vista.classList.remove('hidden');
```

```
    });
</script>
```

VERIFICACIÓN

Ingrese como administrador, abra **Crear nuevo evento**, elija una imagen y confirme que aparece la vista previa. Todavía **no** guarde: el controlador aún no procesa el archivo. La propiedad `imagen_url` se crea en el Ejercicio 4; mientras tanto vale `null` y el bloque «Imagen actual» no se muestra.

commit Copiar

```
$ git add resources/views
$ git commit -m "Agrega campo de imagen y vista previa al formulario de eventos"
```

06 Ejercicio 3 — Guardar la imagen al crear un evento

Ahora el método `store` de `app/Http/Controllers/EventoController.php` debe validar el archivo, moverlo al disco `public` y guardar su ruta. Reemplace el método completo por el siguiente:

app/Http/Controllers/EventoController.php Copiar

```
public function store(Request $request)
{
    $this->authorize('create', Evento::class);

    $datos = $request->validate([
        'titulo' => 'required|string|max:255',
        'descripcion' => 'required|string',
        'fecha' => 'required|date',
        'lugar' => 'required|string|max:255',
        'cupo' => 'required|integer|min:1',
        'precio' => 'required|numeric|min:0',
        'publicado' => ['nullable', 'boolean'],
        'destacado' => ['nullable', 'boolean'],
        'categoria_id' => ['required', 'exists:categorias,id'],
        'etiquetas' => ['nullable', 'array'],
        'etiquetas.*' => ['integer', 'exists:etiquetas,id'],
        // NUEVO: imagen opcional, solo jpg/png/webp y hasta 2 MB
        'imagen' => ['nullable', 'image', 'mimes:jpg,jpeg,png,webp', 'max:2048'],
    ]);

    // validate() devuelve el ARCHIVO (UploadedFile); en la BD solo va la ruta.
    unset($datos['imagen']);

    if ($request->hasFile('imagen')) {
        // Guarda en storage/app/public/eventos/ con un nombre aleatorio único
        // y devuelve la ruta relativa, p. ej. "eventos/Xa9kQ2…T1.jpg"
        $datos['imagen'] = $request->file('imagen')->store('eventos', 'public');
    }

    $datos['slug'] = Str::slug($datos['titulo']);
    $datos['publicado'] = $request->has('publicado') ? 1 : 0;
    $datos['destacado'] = $request->has('destacado') ? 1 : 0;

    // Crea el evento asociado al usuario autenticado
    $evento = $request->user()->eventos()->create($datos);
```

```
    $evento->etiquetas()->sync($request->input('etiquetas', []));

    return redirect()->route('eventos.index')
        ->with('exito', 'Evento creado exitosamente.');
}
```

Análisis línea por línea

Instrucción	Explicación
`'imagen' => ['nullable', …]`	La imagen es opcional. Si viene, debe ser una imagen jpg/png/webp de hasta 2048 KB.
`unset($datos['imagen'])`	Quita el objeto `UploadedFile` del arreglo validado. Si lo dejáramos, Eloquent intentaría guardar el objeto en una columna de texto.
`$request->hasFile('imagen')`	Devuelve `true` solo si llegó un archivo válido (subido sin errores).
`->store('eventos', 'public')`	Genera un nombre aleatorio de 40 caracteres, conserva la extensión correcta y mueve el archivo a la carpeta `eventos` del disco `public`.

NOTA

¿Por qué no usar el nombre original del archivo? Dos personas pueden subir `afiche.jpg` y una sobrescribiría a la otra; además, el nombre original lo controla el usuario y puede contener caracteres problemáticos. Si necesita un nombre legible, use `storeAs('eventos', $slug.'-'.time().'.'.$archivo->extension(), 'public')`.

VERIFICACIÓN

Cree un evento con imagen. Luego verifique:

- En el explorador de archivos: existe un archivo nuevo en `storage/app/public/eventos/`.
- En la base de datos (pgAdmin o `psql`): `SELECT id, titulo, imagen FROM eventos ORDER BY id DESC LIMIT 1;` muestra algo como `eventos/Xa9kQ2…T1.jpg`.
- Intente subir un PDF: debe aparecer un mensaje de error bajo el campo y el evento no se crea.

ADVERTENCIA

En la cartelera la imagen todavía aparecerá **rota**: la vista recibe `eventos/Xa9…jpg` (una ruta) y el navegador la interpreta como `http://localhost:8000/eventos/Xa9…jpg`, que no existe. Lo resolvemos en el siguiente ejercicio.

commit Copiar

```
$ git add app/Http/Controllers/EventoController.php
$ git commit -m "Guarda la imagen del evento en el disco public"
```

07 Ejercicio 4 — Mostrar la imagen con un accessor

Necesitamos convertir la ruta `eventos/Xa9…jpg` en la URL `http://localhost:8000/storage/eventos/Xa9…jpg`. En lugar de repetir esa lógica en cada vista, la encapsulamos en el modelo mediante un **accessor**: un atributo calculado que se lee como si fuera una columna.

4.1 Accessor `imagen_url` en el modelo

En `app/Models/Evento.php` agregue los `use` necesarios y el método `imagenUrl()`:

app/Models/Evento.php Copiar

```
use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Support\Facades\Storage;
use Illuminate\Support\Str;

// ... dentro de la clase Evento, después de $casts:

// Accessor: $evento->imagen_url devuelve la URL pública lista para <img>
protected function imagenUrl(): Attribute
{
    return Attribute::get(function (): ?string {
        if (! $this->imagen) {
            return null;
        }

        // Compatibilidad: registros que guardaron una URL externa completa
        if (Str::startsWith($this->imagen, ['http://', 'https://'])) {
            return $this->imagen;
        }

        return Storage::disk('public')->url($this->imagen);
    });
}
```

DEFINICIÓN

Accessor: método `protected` que devuelve un `Attribute`. Laravel convierte el nombre del método de camelCase a snake_case: `imagenUrl()` se usa como `$evento->imagen_url`. No crea ninguna columna en la base de datos.

4.2 Usar la URL en la cartelera

En `resources/views/eventos/index.blade.php`, dentro de la etiqueta `<x-evento-card>`, cambie el valor del atributo `:imagen`:

resources/views/eventos/index.blade.php Copiar

```
{{-- Antes --}}
:imagen="$evento['imagen']"

{{-- Después --}}
:imagen="$evento->imagen_url"
```

El componente `evento-card` no necesita cambios: ya muestra `<img src="{{ $imagen }}">` cuando hay valor y un ícono de respaldo cuando es `null`. Esa es una ventaja de haber componentizado la tarjeta en la GL04.

4.3 Usar la URL en el detalle

En `resources/views/eventos/show.blade.php`, dentro del bloque **Portada**, cambie la condición y el `src`:

resources/views/eventos/show.blade.php Copiar

```
@if ($evento->imagen_url)
    <img src="{{ $evento->imagen_url }}" alt="{{ $evento->titulo }}" class="h-full w-full object-
cover">
@else
```

VERIFICACIÓN

Recargue la cartelera y el detalle del evento creado en el Ejercicio 3: la imagen debe verse. También puede comprobarlo en Tinker:

php artisan tinker Copiar

```
> $e = App\Models\Evento::whereNotNull('imagen')->latest()->first();
> $e->imagen;
= "eventos/Xa9kQ2…T1.jpg"
> $e->imagen_url;
= "http://localhost:8000/storage/eventos/Xa9kQ2…T1.jpg"
```

commit Copiar

```
$ git add app/Models/Evento.php resources/views
$ git commit -m "Agrega accessor imagen_url y muestra la imagen en las vistas"
```

08 Ejercicio 5 — Reemplazar o quitar la imagen al editar

La edición tiene tres casos posibles, y en ninguno debe quedar un archivo huérfano en el disco:

Caso	Acción del usuario	Qué debe pasar
A	Edita otros campos y no toca la imagen	La imagen actual se conserva
B	Elige un archivo nuevo	Se guarda el nuevo y se borra el anterior
C	Marca «Quitar la imagen actual»	La columna queda en `null` y se borra el archivo

Primero agregue el `use` del facade `Storage` en la parte superior del controlador:

app/Http/Controllers/EventoController.php Copiar

```
use Illuminate\Support\Facades\Storage;
```

Luego reemplace el método `update` completo:

app/Http/Controllers/EventoController.php Copiar

```
public function update(Request $request, Evento $evento)
{
    $this->authorize('update', $evento);
```

```
    $datos = $request->validate([
        'titulo' => 'required|string|max:255',
        'descripcion' => 'required|string',
        'fecha' => 'required|date',
        'lugar' => 'required|string|max:255',
        'cupo' => 'required|integer|min:1',
        'precio' => 'required|numeric|min:0',
        'publicado' => ['nullable', 'boolean'],
        'destacado' => ['nullable', 'boolean'],
        'categoria_id' => ['required', 'exists:categorias,id'],
        'etiquetas' => ['nullable', 'array'],
        'etiquetas.*' => ['integer', 'exists:etiquetas,id'],
        // NUEVO: imagen opcional, solo jpg/png/webp y hasta 2 MB
        'imagen' => ['nullable', 'image', 'mimes:jpg,jpeg,png,webp', 'max:2048'],
    ]);

    unset($datos['imagen']);           // Caso A: no se toca la columna
    $imagenAnterior = $evento->imagen;

    if ($request->hasFile('imagen')) {
        // Caso B: guardar la nueva imagen
        $datos['imagen'] = $request->file('imagen')->store('eventos', 'public');
    } elseif ($request->boolean('quitar_imagen')) {
        // Caso C: dejar el evento sin imagen
        $datos['imagen'] = null;
    }

    $datos['slug'] = Str::slug($datos['titulo']);
    $datos['publicado'] = $request->has('publicado') ? 1 : 0;
    $datos['destacado'] = $request->has('destacado') ? 1 : 0;

    $evento->update($datos);
    $evento->etiquetas()->sync($request->input('etiquetas', []));

    // Borrar el archivo anterior SOLO si la imagen cambió y la BD ya se actualizó
    if ($imagenAnterior && $imagenAnterior !== $evento->imagen) {
        Storage::disk('public')->delete($imagenAnterior);
    }

    return redirect()->route('eventos.index')
        ->with('exito', 'Evento actualizado correctamente.');
}
```

ADVERTENCIA

El `unset($datos['imagen'])` es imprescindible en `update`. Cuando el usuario no elige archivo, el validador puede devolver la clave `imagen` con valor `null`; sin el `unset`, `$evento->update($datos)` **borraría la imagen existente** en cada edición (caso A roto).

NOTA

¿Por qué borramos el archivo anterior **después** de `update()`? Si la actualización fallara (por ejemplo, un `slug` duplicado), el registro seguiría apuntando a la imagen anterior; si ya la hubiéramos borrado, el evento quedaría con una imagen rota. Primero se confirma el cambio en la BD, luego se limpia el disco.

VERIFICACIÓN

Pruebe los tres casos con el mismo evento y, después de cada uno, revise la carpeta `storage/app/public/eventos/`:

- Caso A: cambie solo el título → la imagen se mantiene y no aparecen ni desaparecen archivos.
- Caso B: elija otra imagen → hay un archivo nuevo y el anterior ya no existe.
- Caso C: marque «Quitar la imagen actual» → la carpeta ya no tiene el archivo y la tarjeta muestra el ícono de respaldo.

commit Copiar

```
$ git add app/Http/Controllers/EventoController.php
$ git commit -m "Permite reemplazar y quitar la imagen al editar un evento"
```

09 Ejercicio 6 — Limpiar el archivo al eliminar el evento

Si se elimina un evento, su imagen debe desaparecer también; de lo contrario el disco se llena de archivos que nadie puede ver. Reemplace el método `destroy`:

app/Http/Controllers/EventoController.php Copiar

```
public function destroy(Evento $evento)
{
    $this->authorize('delete', $evento);

    $imagen = $evento->imagen;

    $evento->delete();

    if ($imagen) {
        Storage::disk('public')->delete($imagen);
    }

    return redirect()->route('eventos.index')
        ->with('exito', 'Evento eliminado correctamente.');
}
```

NOTA

Aplicamos el mismo principio que en `update`: primero la base de datos, después el disco. Si `delete()` lanzara una excepción, el archivo seguiría existiendo y el evento también, sin inconsistencias.

VERIFICACIÓN

Elimine un evento con imagen y confirme que su archivo desapareció de `storage/app/public/eventos/`.

commit Copiar

```
$ git add app/Http/Controllers/EventoController.php
$ git commit -m "Elimina la imagen del disco al borrar un evento"
```

10 Trabajo independiente — Foto de perfil del organizador

Aplique lo aprendido a otro modelo: cada usuario podrá subir una foto de perfil (avatar) al registrarse, y esa foto reemplazará al círculo con iniciales que hoy aparece en la tarjeta y en el detalle del evento.

Requisitos mínimos

1. Migración `add_avatar_to_users_table` que agregue la columna `avatar` (`string`, `nullable`) después de `email`.
2. Agregar `avatar` al atributo `#[Fillable([...])]` del modelo `User`.
3. Formulario de registro con `enctype="multipart/form-data"` y un campo de archivo opcional con vista previa.
4. Validación en `RegistroController@store`: imagen jpg/png/webp, máximo 1 MB y `dimensions:min_width=100,min_height=100`. Guardar en la carpeta `avatars` del disco `public`.
5. Accessor `avatar_url` en `User`, siguiendo el patrón de `imagen_url`.
6. Mostrar el avatar (redondo, `object-cover`) en `evento-card` y en el bloque **Organizador** de `show`; si el usuario no tiene avatar, conservar las iniciales.
7. Al menos una prueba Pest que verifique que el avatar se guarda al registrarse.

Desafío opcional (+5 pts extra)

Agregue en la cartelera un filtro **«Solo con imagen»** (checkbox en `partials/filtros`) que use `->when(..., fn ($q) => $q->whereNotNull('imagen'))` en `EventoController@index`.

Entrega

- Repositorio `INF560-2026-02-Apellido` actualizado, con un commit por ejercicio y al menos dos commits para el trabajo independiente.
- En el `README.md`, una sección **GL11** con dos capturas: un evento con imagen en la cartelera y el detalle con el avatar del organizador.

11 Lista de verificación

✓	Comprobación
☐	`php artisan storage:link` ejecutado y documentado en el README
☐	Los formularios de crear y editar tienen `enctype="multipart/form-data"`
☐	El campo de imagen muestra vista previa antes de enviar
☐	Subir un PDF o una imagen mayor a 2 MB muestra un error de validación
☐	La columna `imagen` guarda una ruta relativa (`eventos/…`), no una URL ni el archivo
☐	La imagen se ve en la cartelera y en el detalle mediante `imagen_url`
☐	Editar sin elegir archivo conserva la imagen (caso A)
☐	Reemplazar o quitar la imagen borra el archivo anterior (casos B y C)
☐	Eliminar el evento borra su archivo del disco
☐	Las 5 pruebas de `ImagenEventoTest` pasan

✓	Comprobación
☐	Trabajo independiente (avatar) completo y con prueba

12 Rúbrica de evaluación

Criterio	Logrado	Pts
Disco público	`storage:link` funcionando y documentado; `APP_URL` correcto	5
Formulario	`enctype` en ambos formularios, campo con `accept`, vista previa y opción «Quitar»	10
Validación	Reglas `image`, `mimes`, `max`; mensajes de error visibles bajo el campo	10
Almacenamiento en `store`	Archivo en `storage/app/public/eventos`; la BD guarda solo la ruta relativa	15
Accessor y vistas	`imagen_url` en el modelo; cartelera y detalle muestran la imagen o el respaldo	10
Edición (`update`)	Los tres casos (conservar, reemplazar, quitar) funcionan sin archivos huérfanos	15
Eliminación (`destroy`)	El archivo se borra después de eliminar el registro	5
Pruebas automatizadas	Las 5 pruebas con `Storage::fake()` pasan	10
Trabajo independiente	Avatar del organizador completo según los requisitos, con prueba	15
Repositorio	Un commit por ejercicio, mensajes descriptivos, README con capturas	5
Total		**100**

13 Referencia rápida

Código	Para qué sirve
`$request->hasFile('imagen')`	¿Llegó un archivo válido?
`$request->file('imagen')`	Obtiene el objeto `UploadedFile`
`->store('carpeta', 'public')`	Guarda con nombre aleatorio y devuelve la ruta
`->storeAs('carpeta', $nombre, 'public')`	Guarda con un nombre elegido
`Storage::disk('public')->url($ruta)`	Convierte la ruta en URL pública
`Storage::disk('public')->exists($ruta)`	¿Existe el archivo?
`Storage::disk('public')->delete($ruta)`	Elimina el archivo
`Storage::fake('public')`	Disco falso para pruebas
`UploadedFile::fake()->image('a.jpg', 800, 600)`	Imagen falsa para pruebas

Código	Para qué sirve
`php artisan storage:link`	Crea `public/storage → storage/app/public`

Referencias

Laravel. (2026). *File Storage*. Documentación oficial de Laravel 13. https://laravel.com/docs/filesystem

Laravel. (2026). *Validation — Validating Files*. https://laravel.com/docs/validation#validating-files

Laravel. (2026). *Eloquent: Mutators & Casting — Accessors*. https://laravel.com/docs/eloquent-mutators

MDN Web Docs. (2026). <*input type="file*">. https://developer.mozilla.org/es/docs/Web/HTML/Element/input/file